

Table of Contents

Creating a simple server and client using Java RMI :.....	1
First let me explain the example that I have chosen:	1
Execution and Result:	4
Implementation maintains a persistent state	5
Creating Test	7
Test Results :	7
Github Repo	7

Creating a simple server and client using Java RMI :

First let me explain the example that I have chosen: Mars Rover, Opportunity is sending collected sample's raw data to a server, the Houston computer control in Texas. Houston computer here acts as a server and it has a plethora of remote function at its disposal. One of them is Mission Control. Mission Control packs a strong functionality that is – onboardSampleCheck. Here, client is the Mars Rover, Opportunity. Server is computer control in Houston, Texas and Remote function is Mission Control.

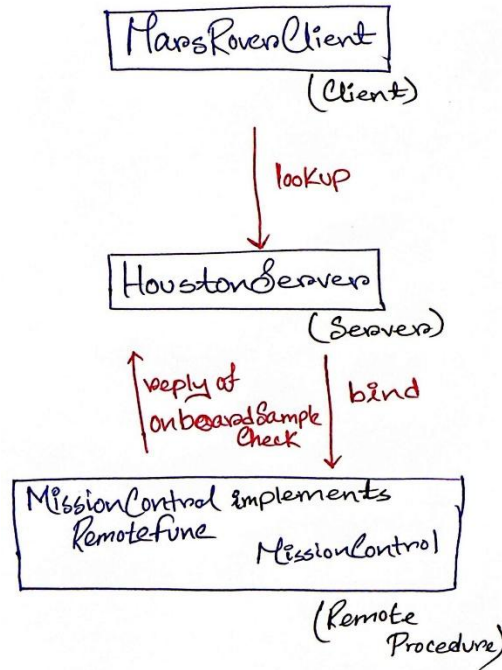


Figure 1: Scenario

First I created the *MissionControl* Interface which extends to Java RMI class *Remote*.

```

© MissionControlImpl_RemoteFunc.java  ⓘ MissionControl.java x
1  import java.rmi.Remote;
2  import java.rmi.RemoteException;
3
4  public interface MissionControl extends Remote { 3 usages 1 implementation
5      String onboardSampleCheck(String sampleId) throws RemoteException; 1 usage 1 implementation
6  }
7
  
```

Then I implemented this interface and created the class *MissionControlImpl_RemoteFunc*. This will work as our remote function. It houses the functionality that is required by client. For this scenario the functionality is – *onboardSampleCheck*. We extended to *UnicastRemoteObject*. It exports a remote object so that the object can accept incoming remote method calls over the network.

```
© MissionControlImpl_RemoteFunc.java × ⓘ MissionControl.java
1  import java.rmi.RemoteException;
2  import java.rmi.server.UnicastRemoteObject;
3
4  public class MissionControlImpl_RemoteFunc extends UnicastRemoteObject implements MissionControl { 2 usages
5
6      protected MissionControlImpl_RemoteFunc() throws RemoteException { 1 usage
7          super();
8      }
9
10     @Override 1 usage
11     public String onboardSampleCheck(String sampleId) throws RemoteException {
12         System.out.println("MissionControlImpl_RemoteFunc: Performing onboard sample check for " + sampleId);
13         return "Sample " + sampleId + " analysis: Composition Stable, Organic traces found.";
14     }
15 }
16
```

Then I created the executable server named *HoustonServer*. It binds *MissionControlImpl_RemoteFunc* with a URL in the network. The server also contains a health check in the form of printing in the console.

```
© MissionControlImpl_RemoteFunc.java  ⓘ HoustonServer.java × ⓘ MissionControl.java
1  import java.rmi.Naming;
2
3  public class HoustonServer {
4      public static void main(String[] args) {
5          try {
6              // Create Remote Function Implementation
7              MissionControlImpl_RemoteFunc missionControlImpl = new MissionControlImpl_RemoteFunc();
8
9              // Bind Remote Object to RMI Registry
10             Naming.rebind("name: rmi://localhost:5000/MissionControl", missionControlImpl);
11
12             System.out.println("HoustonServer is online to take on tasks and delegate");
13         } catch (Exception e) {
14             System.err.println("HoustonServer exception: " + e);
15         }
16     }
17 }
```

Finally the client was made with the Name: *MarsRoverClient*. It is an independently executable file. It looks up the RMI registry with the help of the *MissionControl* RMI by the URL.

```
© MissionControlImpl_RemoteFunc.java  © HoustonServer.java  MarsRoverClient.java x  MissionControl.java
1  import java.rmi.Naming;
2
3  ▶ public class MarsRoverClient {
4  ▶      public static void main(String[] args) {
5          try {
6              // Lookup Remote Object
7              MissionControl missionControl = (MissionControl) Naming.lookup("rmi://localhost:5
8
9              System.out.println("Mars Rover: Sending sample check request...");
10
11              String sampleId = "MARS-SOIL-2025";
12              String result = missionControl.onboardSampleCheck(sampleId);
13
14              System.out.println("Houston replied: " + result);
15
16          } catch (Exception e) {
17              System.err.println("MarsRoverClient exception: " + e);
18          }
19      }
20  }
```

Execution and Result: At first we needed to start the rmiRegistry. We started that using the java command `rmiregistry 5000`.

```
PS C:\uni adelaide\sem 1\dist_comp\lab 1\out\production\lab 1> rmiregistry 5000
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
█
```

Once our registry was updated, we executed server and client separately. This yielded the following results –

```
Run  HoustonServer x  MarsRoverClient x
C:\Users\Esrat71\.jdk\corretto-18.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\In
Mars Rover: Sending sample check request...
Houston replied: Sample MARS-SOIL-2025 analysis: Composition Stable, Organic traces found.
Process finished with exit code 0
```

```
C:\Users\Esrat71\jdk\corretto-18.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3\lib\idea_rt.jar=53744:C:\Program Files\JetBrains\IntelliJ I
HoustonServer is online and delegated tasks to MissionControlImpl.
MissionControlImpl_RemoteFunc: Performing onboard sample check for MARS-SOIL-2025

Process finished with exit code 0
```

Execution summary via console log :

Client started -> “Mars Rover: Sending sample check request...” -> “HoustonServer is online and delegated tasks to MissionControlImpl.” -> “MissionControlImpl_RemoteFunc: Performing onboard sample check for MARS-SOIL-2025” -> “Houston replied: Sample MARS-SOIL-2025 analysis: Composition Stable, Organic traces found.”

Implementation maintains a persistent state

To showcase a persistent state, we have added a list named `getSampleList`. We have also added functions to add and remove samples from the list.

```
MissionControlImpl_RemoteFunc.java  HoustonServer.java  MarsRoverClient.java  MissionControl.java x
1
2 import java.rmi.Remote;
3 import java.rmi.RemoteException;
4 import java.util.List;
5
6 public interface MissionControl extends Remote { 1 usage 1 implementation
7     String onboardSampleCheck(String sampleId) throws RemoteException; no usages 1 implementation
8     void addSample(String sampleId) throws RemoteException; no usages 1 implementation
9     void removeSample(String sampleId) throws RemoteException; no usages 1 implementation
10    List<String> getSampleList() throws RemoteException; no usages 1 implementation
11 }
12 |
```

The implementation of this interface allows the server to add, remove and retrieve samples on behalf of the client. The list persists throughout the life cycle of the connection. `MarsRoverClient` sends rock samples one by one and it is saved in the list by the server. When the client tries to retrieve a sample’s data, server invokes the remote function of getting element from a list to get it and serve it to the client.

```
MissionControlImpl_RemoteFunc.java HoustonServer.java MarsRoverClient.java x MissionControl.java
4
5 public class MarsRoverClient {
6     public static void main(String[] args) {
7         try {
8             MissionControl missionControl = (MissionControl) Naming.lookup( name: "rmi://localhost:
9
10            System.out.println("Mars Rover: Adding samples...");
11            missionControl.addSample( sampleId: "MARS-ROCK-001");
12            missionControl.addSample( sampleId: "MARS-SOIL-007");
13
14            System.out.println("Mars Rover: Requesting sample list...");
15            List<String> samples = missionControl.getSampleList();
16            System.out.println("Current Samples on Houston: " + samples);
17
18            System.out.println("Mars Rover: Performing onboard sample check...");
19            String result = missionControl.onboardSampleCheck( sampleId: "MARS-ROCK-001");
20            System.out.println("Houston replied: " + result);
21
22            System.out.println("Mars Rover: Removing sample MARS-SOIL-007...");
23            missionControl.removeSample( sampleId: "MARS-SOIL-007");
24
25            System.out.println("Mars Rover: Requesting updated sample list...");
26            samples = missionControl.getSampleList();
27            System.out.println("Updated Samples on Houston: " + samples);
28
29        } catch (Exception e) {
```

The console looks like this :

```
Run HoustonServer x MarsRoverClient x
C:\Users\Esrat71\.jdk\corretto-18.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3\lib\idea_rt.jar=55476:C:\Program Files\JetBrains\IntelliJ IDE
HoustonServer is online and delegated tasks to MissionControlImpl_RemoteFunc.
MissionControlImpl_RemoteFunc: Sample MARS-ROCK-001 added.
MissionControlImpl_RemoteFunc: Sample MARS-SOIL-007 added.
MissionControlImpl_RemoteFunc: Performing onboard sample check for MARS-ROCK-001
MissionControlImpl_RemoteFunc: Sample MARS-SOIL-007 removed.
```

```

Run HoustonServer x MarsRoverClient x
Mars Rover: Adding samples...
Mars Rover: Requesting sample list...
Current Samples on Houston: [MARS-ROCK-001, MARS-SOIL-007]
Mars Rover: Performing onboard sample check...
Houston replied: Sample MARS-ROCK-001 analysis: Composition Stable, Organic traces found.
Mars Rover: Removing sample MARS-SOIL-007...
Mars Rover: Requesting updated sample list...
Updated Samples on Houston: [MARS-ROCK-001]

Process finished with exit code 0

```

```

Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\uni adelaide\sem 1\dist_comp\lab 1\out\production\lab 1> rmiregistry 5000
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release

```

Creating Test

We will be doing Junit test here.

- Test 1** -> Adding a sample ID to the server and verify it's reflected in the sample list
Expected result: [MARS-ROCK-001]
- Test 2** -> Removing an existing sample ID from the server and verify it's deleted.
Expected result: Sample list should not contain MARS-SOIL-007
- Test 3** -> Attempting to remove a sample ID that does not exist and verify the server handles gracefully.
Expected result: Server should log "Sample MARS-DUST-999 not found." without exceptions.

Test Results :

<default package> 44 ms ✓ MissionControlImplRemo 44 ms ✓ testOnboardSampleCh 37 ms ✓ testRemoveSample() 2 ms ✓ testSampleListPersisten 2 ms	✓ Tests passed: 5 of 5 tests – 44 ms C:\Users\Esrat71\.jdk\corretto-18.0.2\bin\java.exe ... MissionControlImpl_RemoteFunc: Performing onboard sample check for MARS-ICE-003 MissionControlImpl_RemoteFunc: Sample MARS-SOIL-007 added. MissionControlImpl_RemoteFunc: Sample MARS-SOIL-007 removed.
---	--

Github Repo

https://github.com/sakibesrat71/distributed_computing_uni_adelaide/tree/Java-RMI-feature